

WEEK 7 PRACTICAL

QUESTION 1

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Aim

To write a C program that displays the memory addresses of variables belonging to the **code**, **global**, **static**, **heap**, and **stack** segments and understand the Linux process address-space layout.

Program

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

static int static_global_var = 200;

void sample_function() {
    printf("Inside sample function\n");
}

int main() {
    int stack_var = 300;

    static int static_local_var = 400;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL) {
        perror("malloc");
        return 1;
    }
    *heap_var = 500;
    printf("Memory Addresses:\n\n");

    printf("Code (function)           : %p\n", (void *)sample_function);
    printf("Global variable               : %p\n", (void *)&global_var);
    printf("Static global variable          : %p\n", (void *)&static_global_var);
    printf("Static local variable           : %p\n", (void *)&static_local_var);
    printf("Heap variable                   : %p\n", (void *)heap_var);
    printf("Stack variable                  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output

The exact addresses will be different on every execution because of Linux memory-layout randomization.

Memory Addresses:

```
navaneeth@NAVANEETH:~$ cd ~/OSSP
navaneeth@NAVANEETH:~/OSSP$ mkdir -p Week7
navaneeth@NAVANEETH:~/OSSP$ cd Week7
navaneeth@NAVANEETH:~/OSSP/Week7$ nano memory_segements.c
navaneeth@NAVANEETH:~/OSSP/Week7$ gcc memory_segments.c -o memory_segments
navaneeth@NAVANEETH:~/OSSP/Week7$ ./memory_segments
Memory Addresses:

Code (function)      : 0x5d40f998e1e9
Global variable      : 0x5d40f9991010
Static global variable : 0x5d40f9991014
Static local variable : 0x5d40f9991018
Heap variable        : 0x5d40fe82b010
Stack variable       : 0x7ffe8c656b4c
```

Observation

The program displays the addresses of variables located in different parts of the process address space.

Memory Segment	Example	Purpose
Code / Text	sample_function	Contains executable program instructions
Global / Data	global_var	Stores initialized global variables
Static	static_global_var, static_local_var	Static variables have program-long lifetime and are typically stored in the data or BSS segment depending on initialization.
Heap	heap_var	Used for dynamically allocated memory using malloc()
Stack	stack_var	Stores local variables and function-call information

The addresses demonstrate that different types of data are maintained in different regions of the Linux process address space.

Result

The program was successfully implemented to print the addresses of **code, global, static, heap, and stack variables**. The Linux process memory layout was analyzed based on the addresses obtained.

QUESTION 2

Make use of the `/proc/maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Aim

To study the memory organization of a running Linux process using the `/proc/<PID>/maps` file and memory-analysis commands, and understand virtual memory mappings.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int local_var = 200;

    printf("Process ID: %d\n", getpid());
    printf("Global variable address: %p\n", (void *)&global_var);
    printf("Stack variable address : %p\n", (void *)&local_var);

    printf("\nProcess is running...\n");
    printf("Open another terminal and execute:\n");
    printf("cat /proc/%d/maps\n", getpid());

    printf("\nPress Enter to terminate...\n");
    getchar();

    return 0;
}
```

Output

```
navaneeth@NAVANEETH:~/OSSP/Week7$ nano memory_maps.c
navaneeth@NAVANEETH:~/OSSP/Week7$ gcc memory_maps.c -o memory_maps
navaneeth@NAVANEETH:~/OSSP/Week7$ ./memory_maps
Process ID: 1722
Global variable address: 0x5ee50db7c010
Stack variable address : 0x7ffd2f64f7d4

Process is running...
Open another terminal and execute:
cat /proc/1722/maps

Press Enter to terminate...
```

Useful Memory Analysis Commands

To examine the process mappings:

cat /proc/1722/maps

Sample /proc/1722/maps Output

```
navaneeth@NAVANEETH:~/OSSP/Week7$ cat /proc/1722/map
5ee50db78000-5ee50db79000 r--p 00000000 08:30 80251 /home/navaneeth/OSSP/Week7/memory_maps
5ee50db79000-5ee50db7a000 r-xp 00001000 08:30 80251 /home/navaneeth/OSSP/Week7/memory_maps
5ee50db7a000-5ee50db7b000 r--p 00002000 08:30 80251 /home/navaneeth/OSSP/Week7/memory_maps
5ee50db7b000-5ee50db7c000 r--p 00002000 08:30 80251 /home/navaneeth/OSSP/Week7/memory_maps
5ee50db7c000-5ee50db7d000 rw-p 00003000 08:30 80251 /home/navaneeth/OSSP/Week7/memory_maps
5ee533342000-5ee533363000 rw-p 00000000 00:00 0 [heap]
7ua7ba608000-7ua7ba628000 r--p 00000000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7ua7ba628000-7ua7ba7c0000 r-xp 00028000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7ua7ba7c0000-7ua7ba80e000 r--p 001c0000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7ua7ba80e000-7ua7ba812000 r--p 0020d000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7ua7ba812000-7ua7ba814000 rw-p 00211000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
7ua7ba814000-7ua7ba821000 rw-p 00000000 00:00 0
7ua7ba8df000-7ua7ba8e2000 rw-p 00000000 00:00 0 [vvar]
7ua7ba8e8000-7ua7ba8ea000 rw-p 00000000 00:00 0 [vvar_vclock]
7ua7ba8ea000-7ua7ba8ee000 rw-p 00000000 00:00 0 [vdso]
7ua7ba8ee000-7ua7ba8f0000 rw-p 00000000 08:00 0
7ua7ba8f0000-7ua7ba8f2000 r--p 00000000 00:00 0
7ua7ba8f2000-7ua7ba8f3000 rw-p 00000000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ua7ba8f3000-7ua7ba922000 r-xp 00001000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ua7ba922000-7ua7ba92d000 r--p 00030000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ua7ba92d000-7ua7ba92f000 r--p 0003b000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ua7ba92f000-7ua7ba930000 rw-p 0003d000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ua7ba930000-7ua7ba931000 rw-p 00000000 00:00 0
7f6d2f638000-7f6d2f651000 rw-p 00000000 00:00 0 [stack]
navaneeth@NAVANEETH:~/OSSP/Week7$
```

To view memory statistics:

cat /proc/1722/status

```
navaneeth@NAVANEETH:~/OSSP/Week7$ cat /proc/1722/status
Name:      memory_maps
Umask:     0022
State:     S (sleeping)
Tgid:      1722
Ngid:      0
Pid:       1722
PPid:      793
TracerPid: 0
Uid:       1000    1000    1000    1000
Gid:       1000    1000    1000    1000
FDSize:    256
Groups:    4 24 27 30 46 100 1000
NSTgid:    1722
NSpid:     1722
NSpgid:    1722
NSSid:     793
Kthread:   0
VmPeak:    2768 kB
VmSize:    2768 kB
VmLck:     0 kB
```

To view detailed memory information:

cat /proc/1722/smaps

```

navaneeth@NAVANEETH:~/OSSP/Week7$ cat /proc/1722/smaps
5ee50db78000-5ee50db79000 r--p 00000000 08:30 80251 /home/navaneeth/OSSP/Week7/memory_maps
Size: 4 kB
KernelPageSize: 4 kB
MMUPageSize: 4 kB
Rss: 4 kB
Pss: 4 kB
Pss_Dirty: 0 kB
Shared_Clean: 0 kB
Shared_Dirty: 0 kB
Private_Clean: 4 kB
Private_Dirty: 0 kB
Referenced: 4 kB
Anonymous: 0 kB
KSM: 0 kB
LazyFree: 0 kB
AnonHugePages: 0 kB
ShmemPmdMapped: 0 kB
FilePmdMapped: 0 kB
Shared_Hugetlb: 0 kB
Private_Hugetlb: 0 kB
Swap: 0 kB
SwapPss: 0 kB
Locked: 0 kB
THPeligible: 0

```

You can also use:

pmap 1722

```

navaneeth@NAVANEETH:~/OSSP/Week7$ pmap 1722
1722:  ./memory_maps
00005ee50db78000      4K r---- memory_maps
00005ee50db79000      4K r-x-- memory_maps
00005ee50db7a000      4K r---- memory_maps
00005ee50db7b000      4K r---- memory_maps
00005ee50db7c000      4K rw--- memory_maps
00007f4a7ba608000    132K rw--- [ anon ]
00007f4a7ba628000    160K r---- libc.so.6
00007f4a7ba7c0000    1632K r-x-- libc.so.6
00007f4a7ba80e000    312K r---- libc.so.6
00007f4a7ba812000     16K r---- libc.so.6
00007f4a7ba814000      8K rw--- libc.so.6
00007f4a7ba8df000     52K rw--- [ anon ]
00007f4a7ba8e8000     12K rw--- [ anon ]
00007f4a7ba8ea000      8K rw--- [ anon ]
00007f4a7ba8ee000     16K r---- [ anon ]
00007f4a7ba8f0000      8K r---- [ anon ]
00007f4a7ba8f2000      8K r-x-- [ anon ]
00007f4a7ba8f3000      4K r---- ld-linux-x86-64.so.2
00007f4a7ba922000    188K r-x-- ld-linux-x86-64.so.2
00007f4a7ba92d000     44K r---- ld-linux-x86-64.so.2
00007f4a7ba92f000      8K r---- ld-linux-x86-64.so.2
00007f4a7ba930000      4K rw--- ld-linux-x86-64.so.2
00007f4a7ba931000      4K rw--- [ anon ]
00007fffd2f630000    132K rw--- [ stack ]
total                2768K
navaneeth@NAVANEETH:~/OSSP/Week7$

```

Observation

The `/proc/<PID>/maps` file shows the **virtual memory regions** assigned to a running process.

Common mappings include:

- **Executable/text region** — contains the program's machine instructions.
- **Read-only region** — contains read-only data and other protected mappings.
- **Writable region** — contains initialized and writable program data.

Heap — used for dynamically allocated memory.

- **Shared libraries** — libraries such as the C library are mapped into the process's virtual address space.
- **Stack** — contains local variables and function-call information.
- **VDSO/VVAR** — special kernel-provided mappings used for certain system operations and timing-related functionality.

The permissions shown in `/proc/<PID>/maps` describe what operations are allowed:

r = read

w = write

x = execute

p = private mapping

s = shared mapping

Result

The memory organization of a running Linux process was successfully studied using `/proc/<PID>/maps`, `/proc/<PID>/status`, `/proc/<PID>/smaps`, and `pmap`. The experiment demonstrated how **virtual memory mappings** divide a process's address space into code, data, heap, stack, libraries, and other regions.